

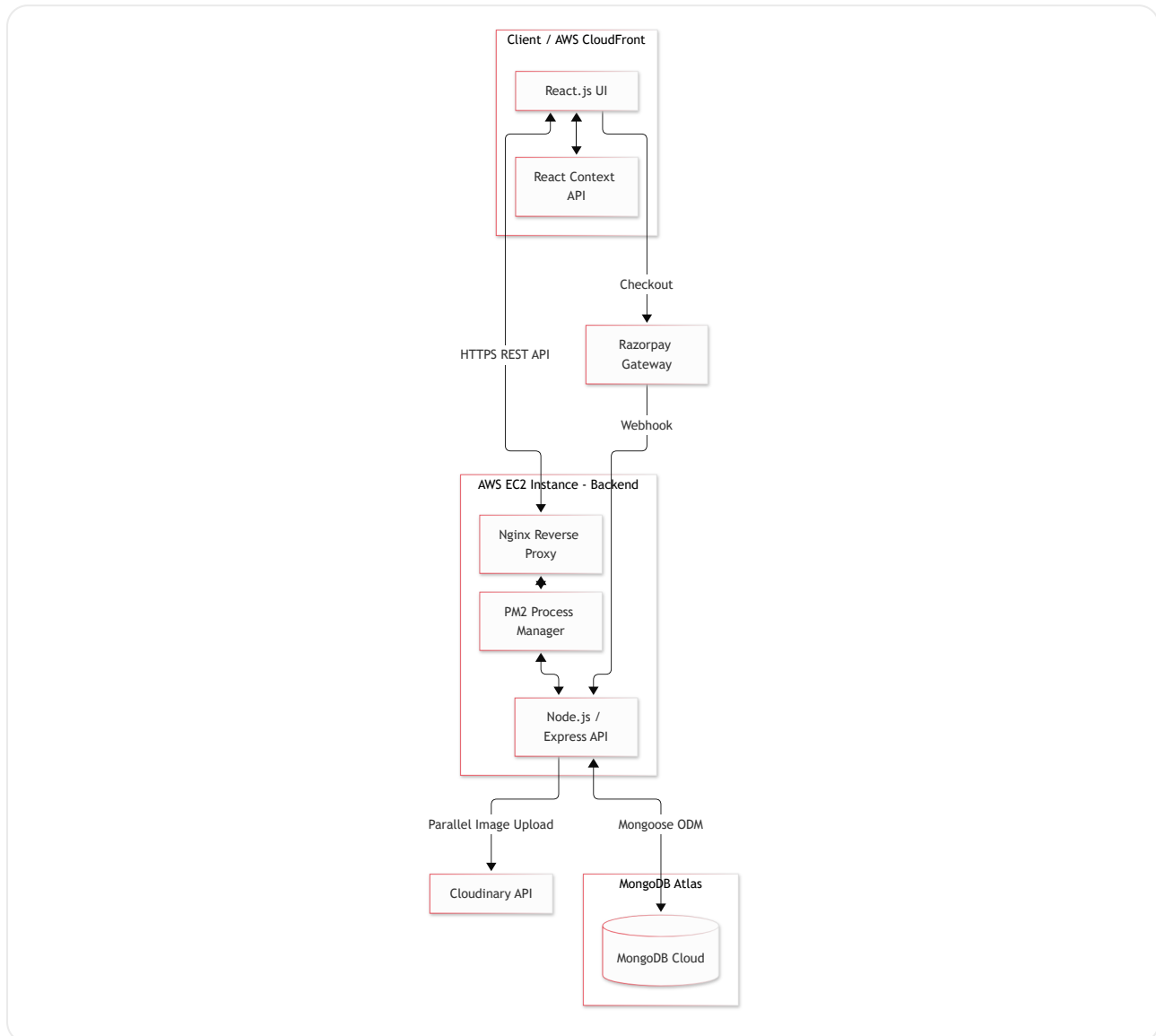
COMPREHENSIVE TECHNICAL DOCUMENTATION

GK'S FITNESS SHOP

Project Overview:

GK's Fitness Shop is a production-grade, full-stack MERN (MongoDB, Express, React, Node.js) e-commerce application. It is engineered to provide a seamless, secure, and highly responsive shopping experience for fitness enthusiasts, featuring real-time inventory management, secure payments, and a fully automated CI/CD cloud deployment architecture.

1. System Architecture Diagram



2. Technology Stack

Frontend (Client-Side)

- **Core:** React.js, React Router DOM
- **State Management:** React Context API (AuthContext, ShopContext)
- **Styling:** Custom SCSS Modules, CSS Grid/Flexbox, Mobile-First Design
- **Icons & Notifications:** Lucide-React, Sonner (Toasts)
- **Data Visualization:** Recharts (Admin Dashboard)

Backend (Server-Side)

- **Core:** Node.js, Express.js
- **Database:** MongoDB Atlas, Mongoose ODM
- **Security:** JWT (JSON Web Tokens), bcryptjs, CORS
- **Media Handling:** Multer (Memory Storage), Cloudinary API

Cloud Infrastructure & DevOps

- **Frontend Hosting:** AWS S3 + AWS CloudFront (CDN)
- **Backend Hosting:** AWS EC2 (Ubuntu)
- **Web Server/Proxy:** Nginx
- **Process Management:** PM2
- **CI/CD:** GitHub Actions (Automated build & deploy)

3. End-to-End Working Process

A. Authentication & Security Flow

1. **Registration:** User enters Name and Email. The backend generates a 6-digit OTP and sends it via Nodemailer/SMTP.
2. **Verification:** The user enters the OTP. Upon success, they proceed to set a secure password.
3. **Password Security:** A real-time regex validator ensures the password has 8+ characters, uppercase, lowercase, numbers, and special symbols.
4. **Data Storage:** Passwords are hashed using bcrypt before being saved to MongoDB. A JWT is issued to the client for stateless session management.
5. **Visibility Toggle:** Custom eye/eye-off toggle components prevent user input errors during login, registration, and profile updates.

B. Catalog & Shopping Flow

1. **Data Fetching:** React fetches products via `/api/products`.
2. **Dynamic UI:** `ProductCard.jsx` reads `countInStock` directly from the database. If stock is 0, the "Add to Cart" button automatically locks and displays "Out of Stock".

3. **Cart Management:** The ShopContext handles cart operations locally, synchronizing item quantities and calculating subtotals.

C. Checkout & Razorpay Payment Pipeline

1. **Order Initialization:** User confirms shipping details and clicks "Pay". React calls the backend to generate a Razorpay Order ID.
2. **Transaction:** Razorpay's modal opens. The user completes the payment securely on the client side.
3. **Cryptographic Verification:** Razorpay returns a `razorpay_payment_id` and `razorpay_signature`. The Node.js backend uses `crypto` and the Razorpay Secret Key to verify the signature authenticity.
4. **Inventory Deduction (Critical):** Upon successful signature verification, the backend loops through the `orderItems` and subtracts the purchased `qty` from the database `countInStock`.

D. Order Management & Cancellation Engine

1. **User Cancellation:** Users can cancel an order if its status is "Pending". The system prompts for a reason.
2. **Admin Cancellation:** Admins can cancel any order, injecting a specific "Cancelled by Admin" message into the tracker.
3. **Stock Restoration Mechanism:** If an order is cancelled, `orderController.js` automatically identifies the items and quantities, and safely adds them *back* to the database stock, preventing permanent inventory loss.

E. Admin Command Center

1. **Dashboard Analytics:** Computes total revenue, active orders, and low-stock alerts, visualizing 7-day revenue trends using `Recharts`.
2. **Inventory Management:** Admins can create, edit, or delete products.
3. **Parallel Image Uploading:** Bypasses slow sequential uploads using `Promise.all`. The backend takes `Multer` RAM buffers and streams up to 5 images simultaneously to `Cloudinary`, returning secure URLs in a fraction of the time.

4. Key Engineering Solutions Highlight

The "True OG" Stock Sync

To prevent frontend/backend data mismatch, the UI strictly reads the source of truth from MongoDB:

```
// ProductCard.jsx
const stockCount = Number(product?.stock ?? product?.countInStock ?? 0);
const isOutOfStock = stockCount <= 0;
```

High-Speed Parallel Image Processing

```
// productController.js
const uploadImagesInParallel = async (files) => {
  const uploadPromises = files.map(file => {
    return new Promise((resolve, reject) => {
      const stream = cloudinary.uploader.upload_stream(
        { folder: 'gks_fitness_products', allowed_formats: ['jpg', 'png', 'webp'] },
        (error, result) => error ? reject(error) : resolve(result.secure_url)
      );
      stream.end(file.buffer);
    });
  });
  return await Promise.all(uploadPromises);
};
```

5. Deployment & CI/CD Architecture

Frontend CI/CD (GitHub Actions -> S3 -> CloudFront)

- Every push to the main branch triggers a GitHub Action.
- The action injects production environment variables securely from GitHub Secrets.
- It builds the React app (`npm run build`) and syncs the `dist/` folder to an AWS S3 Bucket.

- Finally, it executes an AWS CLI command to invalidate the CloudFront cache, ensuring users instantly see the latest updates.

Backend CI/CD (GitHub Actions -> EC2)

- Pushing to the backend main branch triggers an SSH action.
- GitHub securely logs into the AWS EC2 instance using an RSA .pem key.
- The script runs `git pull origin main`, installs new dependencies, and executes `pm2 restart all` to reboot the Node.js server with zero downtime.
- **Nginx Configuration:** Nginx listens on Port 80/443 and proxies all `/api` requests to the local PM2 Node instance running on Port 5000.

Generated for GK's Fitness Shop | Professional E-Commerce Architecture | 2026